

Database Documentation

Playlist & Song Catalog Database

Hazem Ahmed

Entities, Relationships, Schema and Business Rules

Contents

- The Story
- ER Diagram
- Entities and Attributes
- Relationships
- Database Schema
- Business Rules that Don't Show Up on the Diagram

The Story

- Each user has one or many playlists — but each playlist belongs to exactly one user.
- Each playlist has one or many songs, and each song can belong to one or many playlists - a many-to-many relationship.
- Each song belongs to exactly one shared catalog - songs are not owned by any single user or playlist.
- Each playlist–song pairing carries its own data: the date the song was added to that playlist, and the song's order/position within that playlist.

Songs live in a single shared catalog and are visible to every user. A user builds a playlist by selecting existing songs from that catalog and attaching them to their playlist - songs are not uploaded per playlist or per user.

Because the same song can sit inside many different playlists at once, and a playlist obviously holds many songs, the relationship between Playlist and Song is many-to-many. A plain foreign key cannot represent this, so the relationship is implemented through a junction table, PlaylistSong, which resolves the many-to-many relationship

Deleting a user deletes their playlists, since nothing else in the database references a user directly. Deleting a playlist only removes its song associations (its rows in PlaylistSong) - the songs themselves remain untouched in the catalog, since other users' playlists may still reference them. Likewise, deleting a song from the catalog only removes its associations from whatever playlists contained it; it does not delete those playlists.

ER Diagram

The entity-relationship diagram below models the database at the conceptual level

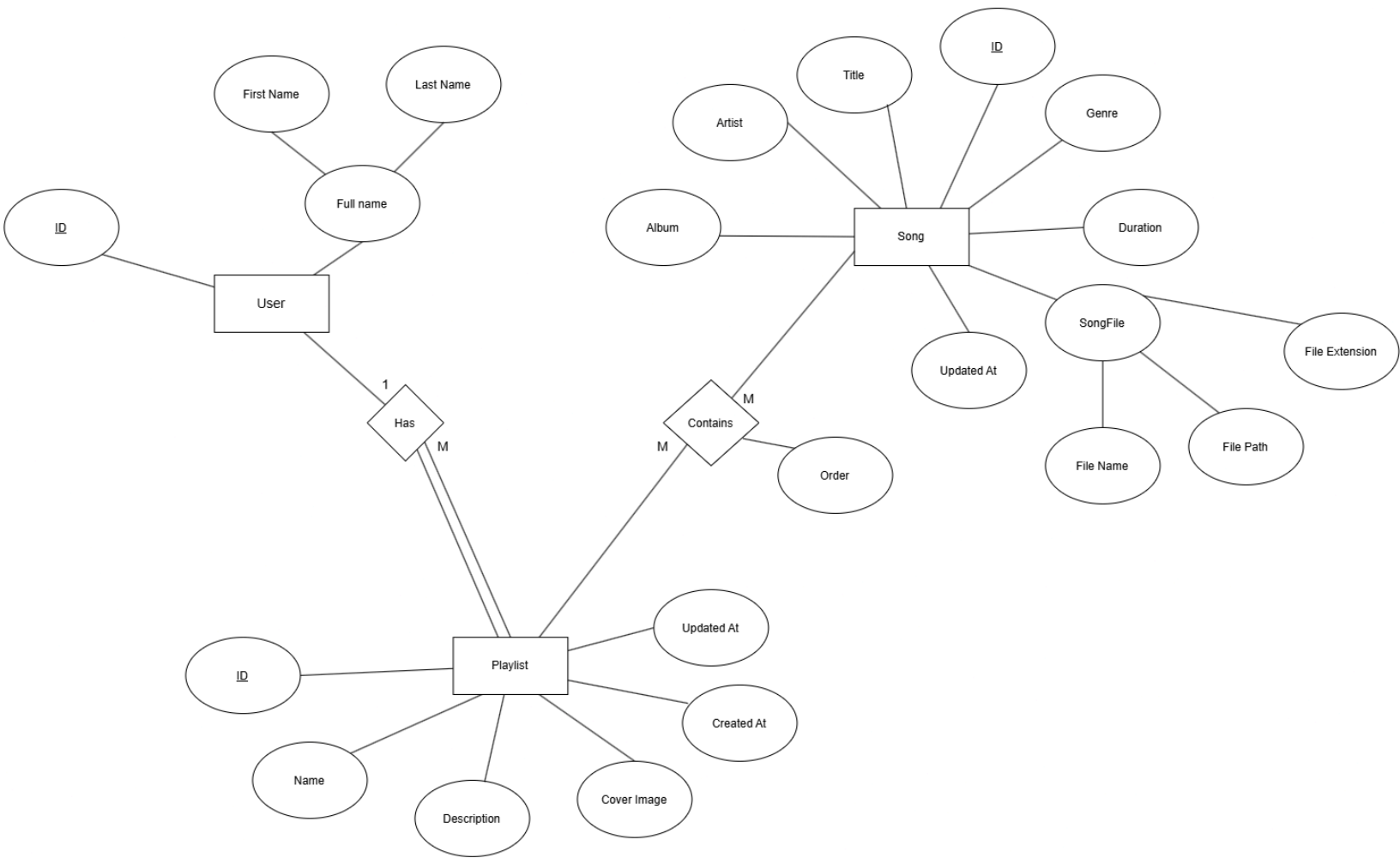


Figure 1 — Entity-Relationship (ER) Diagram

Entities and Attributes

User

A registered account that owns playlists.

Attribute	Type	Required	Constraints	Notes
Id (PK)	string (GUID)	✓	unique	Primary key for the user
FirstName	string	✓	max 50 chars, min 2	
LastName	string	✓	max 50 chars, min 2	
PasswordHash	string	✓	—	Stored hashed, never as plain text
CreatedAt	datetime2	✓	defaults to UTC now	Account creation timestamp

Playlist

A named, user-owned collection of songs.

Attribute	Type	Required	Constraints	Notes
Id (PK)	int	✓	identity/auto-increment	
Name	nvarchar	✓	max 100 chars	Unique per user (application-enforced)
Description	nvarchar	✗	max 500 chars	
CoverImagePath	nvarchar	✗	—	Path to an uploaded cover image; null if none
CreatedAt	datetime2	✓	defaults to UTC now	
UpdatedAt	datetime2	✓	defaults to UTC now	

Song

Catalog metadata for one uploaded audio file, shared by every user.

Attribute	Type	Required	Constraints	Notes
Id (PK)	int	✓	identity/auto-increment	
Title	nvarchar	✓	max 200 chars	
Artist	nvarchar	✓	max 150 chars	
Album	nvarchar	✗	max 150 chars	
Genre	nvarchar	✗	max 100 chars	
Duration	time (TimeSpan)	✓	—	Track length
FileName	nvarchar	✓	max 260 chars	Generated name on disk
FilePath	nvarchar	✓	max 500 chars	Relative path to the stored audio file

Attribute	Type	Required	Constraints	Notes
FileExtension	nvarchar	✓	max 10 chars	e.g. .mp3
FileSize	bigint	✓	—	Bytes
ContentType	nvarchar	✓	max 100 chars	MIME type, e.g. audio/mpeg
UploadedAt	datetime2	✓	defaults to UTC now	

PlaylistSong (junction entity)

Represents one song sitting inside one playlist. This table resolves the Playlist–Song many-to-many relationship into two one-to-many relationships.

Attribute	Type	Required	Constraints	Notes
PlaylistId (PK part 1, FK → Playlist.Id)	int	✓	—	
SongId (PK part 2, FK → Song.Id)	int	✓	—	Composite PK (PlaylistId, SongId) — a song cannot be added to the same playlist twice
AddedAt	datetime2	✓	defaults to UTC now	When the song was attached to this playlist
Order	int	✓	—	Track position within the playlist

Relationships

#	Relationship	Cardinality	Participation
1	User owns Playlist	One-to-many (1 User : N Playlists)	User: partial (a user may have zero playlists). Playlist: total (every playlist must have exactly one owner)
2	Playlist contains Song (via PlaylistSong)	Many-to-many (M Playlists : N Songs)	Playlist: partial (a playlist may have zero songs). Song: partial (a song may belong to zero playlists).

Database Schema

The relational schema below shows the four tables as they exist in the database, with primary keys, foreign keys, and columns.

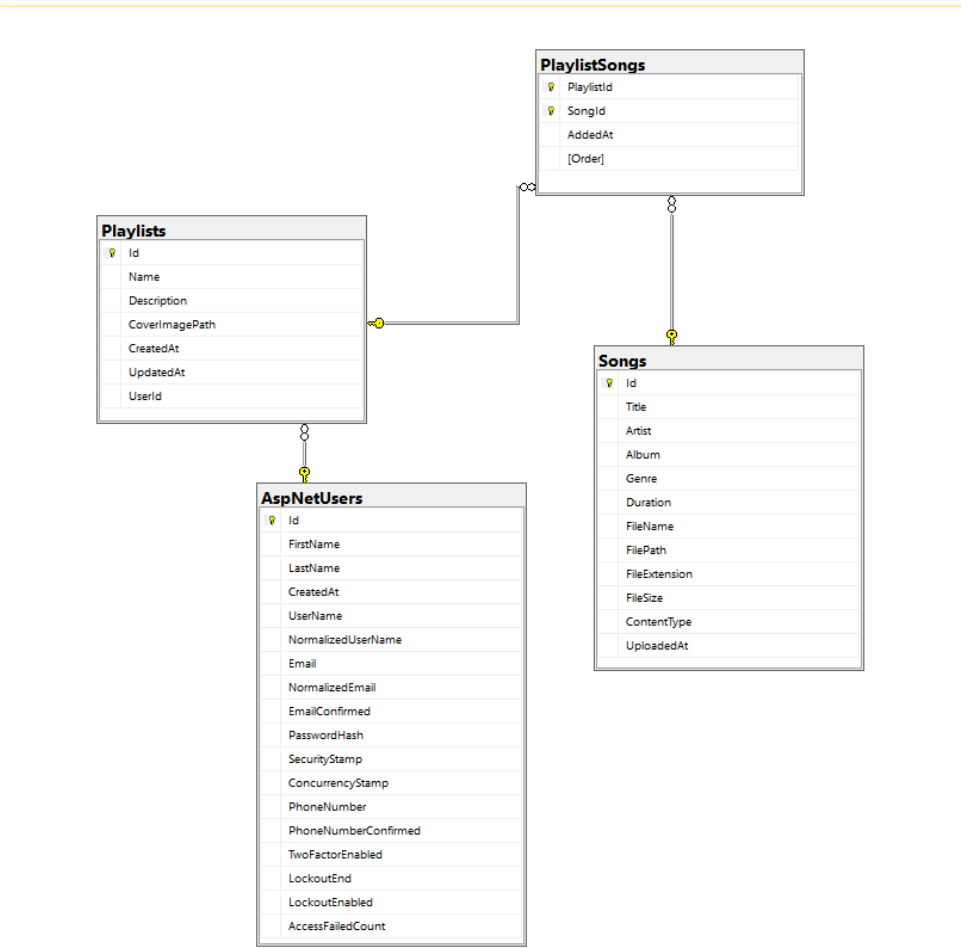


Figure 2 — Database Schema: Playlists, PlaylistSongs, Songs, and Users tables with their keys.

Business Rules that Don't Show Up on the Diagram

These rules are enforced by application logic rather than by a column constraint, so they will not appear on the ER diagram or the schema — they are recorded here separately.

- A playlist name must be unique per user, case-insensitively. Two different users can each have a playlist named “Chill,” but the same user cannot have two playlists with that name. This is checked in application code before insert or update;
- Only certain users are permitted to add, edit, or delete catalog songs; any user may create playlists. This access rule governs which operations a user can perform. Deleting a song or a playlist doesn't only remove database rows — it also removes the associated audio or image file from storage. The database cascade only handles the PlaylistSong join rows; file cleanup is a separate application-level step.